TASKS — Distributed live stacking: a LAN hub, volunteer workers, pages on the site

Source plan: `STARFRONT_DISTRIBUTED_STACKING_PLAN.md` (section numbers below refer to it).

Progress

Phase	Tasks	Area
D	D001–D007	the author's open questions (§9.2), each due before the task that needs it
0	T001–T004	contract and scaffolding (RSC)
1	T101–T111	the hub core (RSC)
2	T201–T208	worker changes (RSC)
3	T301–T310	the hub as a standalone backend (RSC)
4	T401–T417	the Starfront server (SF, plus three RSC tasks)
5	T501–T507	the Starfront desktop (SF)
6	T601–T606	the N.I.N.A. plugin (SF)
7	T701–T705	local mode, docs, CI, the full night

Check progress with `python3 tools/task_progress.py`: per-phase counts, the next task, which decisions are blocking, manual checks left behind, and recent branches in both repositories. `--pending` lists what is left, `--phase N` one phase, `--verify T104` re-runs a task's Verify command, and `--verify-done` re-runs every ticked task's (exits 1 if any no longer passes).

The same facts are on a private web page, **Distributed Stacking Progress** (https://claude.ai/artifact/WJummaRZgi5Z3v6asiT9MX), readable from a phone. It shows the state at its last refresh: after ticking boxes, ask Claude to refresh the progress page (`python3 tools/task_progress.py --export DIR`, then the two files are uploaded to it). A source older than a week is marked stale.

Phase D — decisions the author owes (§9.2)

- [ ] D001 — Library frames with two backends: one library per backend (the plan's default) or fed from the user's `/files` darks
  Manual: decide before T405 and T602. Record the answer as an open point in RSC `docs/api/README.md` (T001 leaves a placeholder).
- [ ] D002 — Who may bind a collaboration to a hub: the hub owner only, or coordinators on the hub's `allowedCoordinators` list (the plan)
  Manual: decide before T403.
- [ ] D003 — Visibility default for hub projects: `unlisted` (the plan) or `public`
  Manual: decide before T403.

- [ ] D004 — What closing or deleting a hub-backed collaboration does (keep masters? return raw to contributors? the 7-day delay?)
  Manual: decide before T704.

- [ ] D005 — `collab-worker self-update` for headless Mac and Linux volunteers in v1, or re-running the install
  Manual: decide before T207.

- [ ] D006 — TLS on the LAN: off by default (the plan) or on with a pinned self-signed certificate from day one
  Manual: decide before T102.

- [ ] D007 — The ported pages: relicensed MPL-2.0 inside Starfront, or kept under `server/web/stack/` with their own notice
  Manual: decide before T408.

Phase 0 — contract and scaffolding (RSC, §Phase 0)

- [ ] T001 — Contract 1.27.0: the hub profile and open points 109–114
  Files: RSC `docs/api/openapi.yaml`, `docs/api/README.md`, `docs/api/hub-profile.json`, `docs/api/queues.md`, `docs/api/s3-layout.md`, `docs/api/status.schema.json`, `backend/lambdas/shared/contract.json`
  Do: §0.A: the hub profile (operations a hub implements and does not); bearer kinds incl. `sfg_`; heartbeat `contract` / `catalogue` / `catalogue_path_ok` / `idle_policy` / `busy` and `min_contract`, 409 `contract_mismatch` (in `ErrorCode`, the plugin's `ApiCodes`); relative part URLs; the catalogue endpoint; "On the hub" in `queues.md` and `s3-layout.md`. Placeholders for D001. `info.version` 1.27.0, additive.
  Done when: a hub can be built against the contract alone.
  Verify: `npx --yes @redocly/cli@latest lint docs/api/openapi.yaml && (cd backend && node scripts/gen-contract.js) && git diff --exit-code backend/lambdas/shared/contract.json && node scripts/check-contract.mjs`

- [ ] T002 — The `collab-hub` crate: scaffold and workspace dependencies
  Files: RSC `worker/Cargo.toml`, `worker/Cargo.lock`, `worker/crates/hub/`
  Do: §0.B: `collab-hub` lib + binary (`--help` only), `[lints] workspace = true`; workspace deps `axum`, `tokio`, `tower-http`, `rusqlite` (bundled), `hmac`, `ulid`, `rand`, `hex`, `mime_guess`, `sysinfo`. `worker-ci` builds it on three OSes with no workflow change.
  Done when: the crate builds, is linted, and the dependency-skew check still passes.
  Verify: `cd worker && cargo build -p collab-hub && cargo clippy -p collab-hub --all-targets -- -D warnings && bash scripts/check-dep-skew.sh && bash scripts/check-no-local-patch.sh`

- [ ] T003 — A shared `wire` module for the RPC shapes
  Files: RSC `worker/crates/worker/src/wire.rs`, `worker/crates/worker/src/daemon/ports.rs`, `worker/crates/worker/src/daemon/child.rs`, `worker/crates/worker/src/lib.rs`
  Do: move the typed route structs of `ports.rs` and the request/reply op enums of `child.rs` into `wire.rs`, re-exported from both; no behaviour change.

Done when: the hub can depend on one definition of every wire shape.
Verify: `cd worker && cargo test -p collab-worker`

- [ ] T004 — `check-contract.mjs`: the hub component (static, warning mode)
  Files: RSC `scripts/check-contract.mjs`, `scripts/lib/hub.mjs`
  Do: §0.C: read `.route("…")` strings and error codes from `worker/crates/hub/src/routes/*.rs`, compare with `hub-profile.json` and the contract; missing profile routes are a warning until T307 makes them an error; a route or code the profile does not name is an error now.
  Verify: `node scripts/check-contract.mjs`

Phase 1 — the hub core (RSC, §Phase 1)

- [ ] T101 — A generic store conformance suite, `DirStore` held to it
  Files: RSC `worker/crates/engine/src/test_support.rs`, `worker/crates/engine/tests/`
  Do: §1.4: `store_conformance<S: Store>(make)` from the `Store`-only assertions of `tests/{lease,accumulate,pipeline_gold}.rs`; a `FaultStore` wrapper for the fault plans; a test running it over `DirStore`.
  Done when: any `Store` can be checked by one call, and `DirStore` passes.
  Verify: `cd worker && cargo test -p collab-engine`

- [ ] T102 — Hub config, `check`, the error envelope and the app skeleton
  Files: RSC `worker/crates/hub/src/{main,config,app,error}.rs`, `worker/crates/hub/tests/`
  Do: §1.2: `collab-hub.toml` + `COLLAB_HUB_*`, `check` (data root writable and free space, XPSD folder probe, exit non-zero on a problem), `HubError` with the contract envelope, the layer stack and a health route; TLS per D006.
  Done when: `collab-hub check` reports every missing or invalid setting, and the hub serves `/api/hub/v1/health`.
  Verify: `cd worker && cargo test -p collab-hub`

- [ ] T103 — The SQLite layer and schema
  Files: RSC `worker/crates/hub/src/db/`, `worker/crates/hub/tests/`
  Do: §0.2, §1.3: one writer behind a mutex plus readers, WAL, `busy_timeout`, `with_tx` (`BEGIN IMMEDIATE`), `include_str!` migrations recorded in `_migrations`, the schema of §1.3; the README note on `rusqlite` over `sqlx`.
  Verify: `cd worker && cargo test -p collab-hub db`

- [ ] T104 — `SqliteQueues`
  Files: RSC `worker/crates/hub/src/queue.rs`, `worker/crates/hub/tests/queue.rs`
  Do: §1.6: the `daemon/fake/mod.rs` semantics on SQLite: visibility, receipts, redrive at 5 to `-dlq`, delayed send, attributes, retention; a `Notify` per queue for long polls; the fake's queue tests ported.
  Verify: `cd worker && cargo test -p collab-hub --test queue`

- [ ] T105 — `SqliteStore`: reads and the prepare transitions
  Files: RSC `worker/crates/hub/src/store/`
  Do: §1.4: every read; T4, T4b, T4c, T4d, T13, T16; `set_photometric_anchor_if_absent`; the `project_status` column kept in step.
  Verify: `cd worker && cargo test -p collab-hub store`

- [ ] T106 — `SqliteStore`: leases, checkpoints and gold
  Files: RSC `worker/crates/hub/src/store/`, `worker/crates/hub/tests/store_conformance.rs`
  Do: `StreamLeases`, `commit_checkpoint` (incl. mark-only and preview-only forms), `record_tier1`, `mark_in_gold`, `publish_gold`, `set_stream_preview`, `reserve_gold_seq`, job creation with the `active_gold_job` guard.
  Done when: the T101 suite passes over `SqliteStore`.
  Verify: `cd worker && cargo test -p collab-hub --test store_conformance`

- [ ] T107 — `SqliteStore::complete_prepare` with the Lambda-only parts
  Files: RSC `worker/crates/hub/src/store/frames.rs`, `worker/crates/hub/tests/`
  Do: T5–T8, T8r, T14, T15, T17 plus `prepare-results.js`'s rotation-from-first-light, `retireIfEmptied`, the gold threshold trigger, the debounced calibration `reprepare` job, the `accumulate` send.
  Verify: `cd worker && cargo test -p collab-hub`

- [ ] T108 — Objects and record fixtures
  Files: RSC `worker/crates/hub/src/objects.rs`, `worker/crates/hub/tests/record_fixtures.rs`
  Do: §1.5: `DirObjects` over `<data_root>/objects`; every `docs/api/examples/records/engine/*.json` loaded into `SqliteStore` and read back byte-identical.
  Verify: `cd worker && cargo test -p collab-hub --test record_fixtures`

- [ ] T109 — The worker-route rules made generic over `&dyn Store`
  Files: RSC `worker/crates/worker/src/daemon/fake/api.rs`, `worker/crates/worker/src/daemon/fake/`
  Do: §1.7: the five route functions take `&dyn Store` and a `RouteCtx` (clock, queue sender, gold trigger); the fake keeps calling them over `DirStore`.
  Done when: the daemon tests pass unchanged.
  Verify: `cd worker && cargo test -p collab-worker --test daemon_prepare --test daemon_work --test daemon_loop --test daemon_e2e`

- [ ] T110 — Worker routes, the RPC and worker auth on the hub
  Files: RSC `worker/crates/hub/src/routes/{worker,rpc}.rs`, `worker/crates/hub/src/auth.rs`, `worker/crates/hub/tests/worker_routes.rs`
  Do: §1.7: `/api/worker/*` over the T109 functions (heartbeat answers `min_contract`, `server_time`, 409 below); `/hub/v1/store/{op}` (the closed list only), `/hub/v1/objects/{key}` (Range, HEAD, streamed PUT, listing; `raw/` refused), `/hub/v1/queue/{op}`; `rscw_` tokens and `collab-hub token`.
  Verify: `cd worker && cargo test -p collab-hub --test worker_routes`

- [ ] T111 — `collab-hub import` and the hub variant of `daemon_e2e` (ignored until T203)
  Files: RSC `worker/crates/hub/src/main.rs`, `worker/crates/hub/src/import.rs`, `worker/crates/worker/tests/daemon_e2e.rs`
  Do: import a `simulate --synthetic --out DIR` tree (records and objects); `collab_hub::app::serve(cfg)` returning the bound address; the `--hub` scenario written and `#[ignore]`d.
  Verify: `cd worker && cargo test -p collab-hub && cargo clippy --workspace --all-targets -- -D warnings`

Phase 2 — worker changes (RSC, §Phase 2)

- [ ] T201 — Worker config names for a hub
  Files: RSC `worker/crates/worker/src/daemon/config.rs`, `docs/runbooks/worker-config.md`, `backend/terraform/worker-config.tf` (the name list only)
  Do: §2.2: `COLLAB_HUB_URL`, `COLLAB_HUB_CERT_SHA256`, `COLLAB_IDLE_POLICY`, `COLLAB_BUSY_FILE`, `COLLAB_WORK_WINDOW`, `COLLAB_WORKER_TOKEN_FILE`; AWS names not required with a hub; a mixed configuration refused by `config check`.
  Verify: `cd worker && cargo test -p collab-worker && cd ../backend/terraform && terraform init -backend=false -input=false >/dev/null && env -u AWS_PROFILE -u AWS_ACCESS_KEY_ID -u AWS_SECRET_ACCESS_KEY -u AWS_SESSION_TOKEN terraform test`

- [ ] T202 — `daemon/hub.rs`: `HubStore`, `HubObjects`, `HubQueues` and the `hub` feature
  Files: RSC `worker/crates/worker/src/daemon/hub.rs`, `daemon/start.rs`, `daemon/mod.rs`, `worker/crates/worker/Cargo.toml`, `worker/crates/worker/tests/hub_ports.rs`
  Do: §2.1: `HubStore` in `DdbStore`'s shape over the RPC; `LocalBlobs` / `LocalQueues` at the hub origin; backend selection in `start.rs`; a `hub` feature needing only `ureq`.
  Verify: `cd worker && cargo test -p collab-worker --test hub_ports && cargo build -p collab-worker --no-default-features --features hub,synthetic`

- [ ] T203 — `daemon_e2e --hub` un-ignored: one worker, crashes, two workers
  Files: RSC `worker/crates/worker/tests/daemon_e2e.rs`
  Done when: the daemon through the hub produces `simulate`'s output bit for bit, also with crashes and with two workers.
  Verify: `cd worker && cargo test -p collab-worker --test daemon_e2e`

- [ ] T204 — Enrolment: hub enrol codes and `collab-worker enrol`
  Files: RSC `worker/crates/worker/src/enrol.rs`, `worker/crates/worker/src/main.rs`, `worker/crates/hub/src/routes/admin.rs`, `worker/crates/hub/src/auth.rs`, tests in both crates
  Do: §2.3, §3.3: `POST /api/hub/v1/enrol-codes` (admin token) and `POST /api/hub/v1/enrol` (code-gated, rate-limited, one use, expiry); `enrol` writes `collab-worker.toml` and a 0600 `worker-token`, runs `config check`, is idempotent and revokes the old token.
  Verify: `cd worker && cargo test -p collab-worker --test hub_ports --test cli && cargo test -p collab-hub`

- [ ] T205 — Capabilities, catalogue probe, Windows RAM and the version handshake
  Files: RSC `worker/crates/worker/src/daemon/{capabilities,housekeeping,run}.rs`, `worker/crates/worker/build.rs`, `worker/crates/worker/Cargo.toml`
  Do: §2.4: `contract` from `openapi.yaml` at build time; `probe_xpsd` at start and every 5 min; RAM through `sysinfo` on every OS; 409 `contract_mismatch` drains and exits 3.
  Verify: `cd worker && cargo test -p collab-worker`

- [ ] T206 — Pause while imaging and work windows
  Files: RSC `worker/crates/worker/src/daemon/run.rs`, a shared window parser moved from `worker/crates/uploader/src/`, tests
  Do: §2.4: a `Gate` before each receive; `busy` from `COLLAB_BUSY_FILE`, a `NINA.exe` process check,

or `COLLAB_WORK_WINDOW`; jobs in flight finish; the heartbeat reports why the worker is idle.
Verify: `cd worker && cargo test -p collab-worker && cargo test -p collab-uploader`

- [ ] T207 — `collab-worker service install|uninstall|start|stop|status`
Files: RSC `worker/crates/worker/src/service.rs`, `worker/scripts/launch-agent.sh`, `docs/runbooks/worker-daemon.md`
Do: §2.5: LaunchAgent as the script writes it; `systemd --user` with `RestartPreventExitStatus=3`; Windows logon Scheduled Task by default, `--as-service` via `windows-service`; exit 3 is "do not restart" everywhere; self-update per D005.
Verify: `cd worker && cargo test -p collab-worker --test cli && cargo clippy --workspace --all-targets -- -D warnings`

- [ ] T208 — Two real workers on the home network drain a hub
Manual: `collab-hub run` on the Mac; `collab-worker enrol` and `service install` on the Mac and the Windows PC (XPSD over SMB); `collab-hub status` shows both with `catalogue: ready`; unmount the share on Windows and see `missing` within 5 min; touch the busy file and see polling stop.

Phase 3 — the hub as a standalone backend (RSC, §Phase 3)

- [ ] T301 — Frame ingress: multipart init, part PUT, complete; library frames
Files: RSC `worker/crates/hub/src/routes/frames.rs`, `worker/crates/hub/src/store/frames.rs`, tests
Do: §3.1: T1 (sentinel, quota), resume, duplicate; parts written at offset; ETags; `bad_etags`, `upload_incomplete`, `size_mismatch`; magic check; T2/T2b and the prepare send; expired uploads aborted; `library/frames/*` under `lib/`.
Verify: `cd worker && cargo test -p collab-hub`

- [ ] T302 — Contributor routes
Files: RSC `worker/crates/hub/src/routes/projects.rs`, tests
Do: §3.2: resolve-join-code, project, join, membership, rigs (fingerprint per `contract-examples.json`), `filters/normalize`, frames list/get/withdraw, status, masters, events, `aliases.json`; signed URLs for private projects.
Verify: `cd worker && cargo test -p collab-hub`

- [ ] T303 — Admin surface for the desktop
Files: RSC `worker/crates/hub/src/routes/admin.rs`, tests
Do: §3.3: admin tokens, fleet, pause, projects with queue depths and disk use, standalone project create and update, hub status, worker revoke.
Verify: `cd worker && cargo test -p collab-hub`

- [ ] T304 — Public artefacts and the publish signal
Files: RSC `worker/crates/hub/src/routes/public.rs`, `worker/crates/hub/src/objects.rs`
Do: §3.4: `/public/{project_id}/…` with `s3-layout.md` cache headers and content types, `..` refused; a `status.json` write queues a push (consumed in T415).
Verify: `cd worker && cargo test -p collab-hub`

- [ ] T305 — Background tasks: sweep, gold scheduler, retention, DLQ drain, free-space floor
  Files: RSC `worker/crates/hub/src/tasks/`, tests
  Do: §3.5: ports of `sweep`, `gold-scheduler`, `retention-sweeper` (with `project-purge.js`), `prepare-dlq`, `gold-dlq`; the `reg/` cache pruned below `free_gb_floor`.
  Verify: `cd worker && cargo test -p collab-hub`

- [ ] T306 — The catalogue service and the worker's `HubCatalogs`
  Files: RSC `worker/crates/hub/src/catalogue.rs`, `worker/crates/worker/src/daemon/catalog.rs`, tests
  Do: §3.6: one cone search per project and `grid.spec()` on the hub; 202 `building`; rebuild on `grid_rev`; the worker fetches, falls back to its own XPSD on 404 and `PUT`s the result back.
  Verify: `cd worker && cargo test -p collab-hub && cargo test -p collab-worker`

- [ ] T307 — `collab-hub simulate` and the strict contract check
  Files: RSC `worker/crates/hub/src/main.rs`, `worker/crates/hub/src/simulate.rs`, `scripts/lib/hub.mjs`
  Do: §3.7, §0.C: a seeded hub on a free loopback port printing `{url, admin_token, enrol_code}`; the contract check drives every profile operation and validates answers; missing profile routes become an error.
  Verify: `node scripts/check-contract.mjs`

- [ ] T308 — `scripts/hub-e2e.sh`, `e2e-check`, the uploader's hub test and the CI job
  Files: RSC `scripts/hub-e2e.sh`, `worker/crates/hub/src/e2e_check.rs`, `worker/crates/uploader/tests/e2e_hub.rs`, `.github/workflows/worker-ci.yml`
  Do: §3.7: hub, one or two daemons, `collab-uploader` through the real multipart path, compare with `simulate` bit for bit, browser smoke when Chrome exists; output under `~/Library/Caches/collab-hub-e2e/`; a `hub-e2e` CI job (ubuntu, macos).
  Verify: `bash scripts/hub-e2e.sh --only all`

- [ ] T309 — Backup and the hub runbook
  Files: RSC `worker/crates/hub/src/backup.rs`, `docs/runbooks/starfront-hub.md`, `CLAUDE.md`, `AGENTS.md`
  Do: §9.1: `collab-hub backup DIR` (`VACUUM INTO` plus masters and public); a first runbook (install, config, SMB share, firewall, backup); the repo guidance gains the hub crate and its commands.
  Verify: `cd worker && cargo test -p collab-hub`

- [ ] T310 — Real data through a standalone hub
  Manual: the Elephant's Trunk subset through `collab-uploader` to the Mac hub, the Windows worker preparing against the Mac's SMB share; previews rendered by `website/scripts/browser-smoke.mjs --sim <hub>/objects/public`.

Phase 4 — the Starfront server (§Phase 4)

- [ ] T401 — Server schema for hubs and stacking; remove `server/stacks.py`
  Files: SF `server/store.py`, `server/stacks.py` (deleted), `tools/check_livestack.py` (its server cases), `tools/check_server.py`
  Do: §4.1: `hubs`, `stack_*` tables and migrations; delete the orphaned router and the check cases

that drove it.
Verify: `python3 tools/check_server.py && python3 tools/check_livestack.py`

- [ ] T402 — Hub registration, heartbeat, grants and the change feed
Files: SF `server/hubs.py`, `server/app.py` (mount), `tools/check_server.py`
Do: §0.3, §4.2: register (shown-once `sfh_` token and grant secret), list, revoke, heartbeat; HMAC grant mint and verify; `pull?since=` long poll over `stack_changes`.
Verify: `python3 tools/check_server.py`

- [ ] T403 — The project binding and the listing
Files: SF `server/app.py`, `server/stack/defaults.py`, `tools/check_server.py`
Do: §4.3: the `stacking` block (cloud or hub, per D002 and D003), `stack_projects` minted on bind, contract policy defaults, `_stacking_of` for old payloads, the listing's `stacking` object with the grant under the join-code secrecy rule, old fields kept.
Verify: `python3 tools/check_server.py`

- [ ] T404 — The front door: envelope, auth, projects, membership, rigs, header allow-list
Files: SF `server/stack/{envelope,auth,projects,rigs,header}.py`, `server/app.py`, `tools/check_server.py`
Do: §4.4: the Collab envelope with no 403 or 404; `sfg_` and cookie auth; resolve-join-code, project, join, membership, rigs (the contract's fingerprint example), `InitHeader`.
Verify: `python3 tools/check_server.py`

- [ ] T405 — The front door: frames multipart and the library
Files: SF `server/stack/{frames,library}.py`, `tools/check_server.py`
Do: §4.4: init, resume, duplicate, part `PUT` at `/stack/s3/{upload_id}/{n}` written at offset, complete, list, get, withdraw; library frames per D001; 429 `quota_exceeded` below `KEEP_FREE`.
Verify: `python3 tools/check_server.py`

- [ ] T406 — The relay: frames for the hub, ranged bytes, ack, retention
Files: SF `server/hubs.py`, `server/files.py` (the `collectFiles` registration), `tools/check_server.py`
Do: §4.2: list uploaded frames, `Range` bytes, ack, deletion after `STARFRONT_STACK_RELAY_KEEP_DAYS`, a `collectFiles` project's frames also on `/files`.
Verify: `python3 tools/check_server.py`

- [ ] T407 — Push ingestion and `/stack/public`
Files: SF `server/hubs.py`, `server/app.py`, `tools/check_server.py`
Do: §4.2, §4.5: multipart push of records and files, atomic writes, per-hub byte budget; `/stack/public/{project_id}/…` with the hub's cache headers; verdicts shown by `frames?mine=1`.
Verify: `python3 tools/check_server.py`

- [ ] T408 — The ported pages, their routes and the CSP
Files: SF `server/web/stack/`, `server/app.py`, `tools/check_csp.py`, `README.md` (CRLF)
Do: §4.5: project, filter, combined pages and their JS/CSS from RSC `website/`, re-accented; the five page routes; generated `site-config.js`; the CSP middleware on `/stack/*`; licence notice per D007; the README's "no web interface" sentence replaced by the decision.
Verify: `python3 tools/check_csp.py && python3 tools/check_server.py`

- [ ] T409 — "My projects": the cookie dashboard
  Files: SF `server/web/stack/dashboard.html`, `server/web/stack/js/pages/dashboard.js`, `server/stack/auth.py`, `tools/check_server.py`
  Do: §4.5: `/files/signin` cookie; `auth/me`; the user's projects with the hub's heartbeat status; "My frames" with verdicts and CSV; 401 without the cookie.
  Verify: `python3 tools/check_server.py && python3 tools/check_csp.py`

- [ ] T410 — Worker and hub binary downloads on the server
  Files: SF `server/app.py`, `server/downloads/collab-worker/.gitkeep`, `.gitignore`, `tools/check_server.py`
  Do: §4.6: `/download/collab-worker/{name}` and `/download/collab-hub/{name}` under strict regexes; `latest.json` and `latest.json.sig` served as files; archives git-ignored.
  Verify: `python3 tools/check_server.py`

- [ ] T411 — Release archives and the signed manifest (RSC)
  Files: RSC `worker/scripts/sign-manifest.sh`, `worker/scripts/build-manifest.sh`, `.github/workflows/worker-release.yml`
  Do: §4.6: per-OS archives of `collab-worker` and `collab-hub`; a manifest in rapid-stacker's shape plus `contract`; an Ed25519 signature by an offline key (key generation is the author's; the workflow never runs on its own).
  Verify: `bash worker/scripts/build-manifest.sh --dry-run`

- [ ] T412 — Discord notifications
  Files: SF `server/notify.py`, `server/hubs.py`, `server/app.py`, `tools/check_server.py`
  Do: §4.7: the owner-only `discordWebhookUrl`; `gold_published` and `member_joined`; de-duplicated through `stack_notified`; a fake webhook in the check.
  Verify: `python3 tools/check_server.py`

- [ ] T413 — `run_local.py --lan`
  Files: SF `server/run_local.py`, `tools/check_local_server.py`
  Do: §4.9: bind `0.0.0.0` only with `--lan`, refused on a real data folder or one with `discord.env`, a loud warning and the LAN URL.
  Verify: `python3 tools/check_local_server.py`

- [ ] T414 — `tools/check_stack_relay.py`: a fake hub against the server
  Files: SF `tools/check_stack_relay.py`
  Do: §4.8: register, bind, upload, pull, fetch, ack, push and page render in one isolated run.
  Verify: `python3 tools/check_stack_relay.py`

- [ ] T415 — The hub's Starfront sync task (RSC)
  Files: RSC `worker/crates/hub/src/{starfront,publish}.rs`, `worker/crates/hub/src/tasks/`, `worker/crates/hub/src/auth.rs`, tests
  Do: §0.4: register credentials, heartbeat, the change-feed pull into `SqliteStore`, frame fetch with `Range` and ack, push on publish, grant verification, `identities`; a fake Starfront server in the tests.
  Verify: `cd worker && cargo test -p collab-hub`

- [ ] T416 — `check-contract.mjs`: the Starfront front-door component (RSC)
  Files: RSC `scripts/lib/starfront.mjs`, `scripts/check-contract.mjs`

Do: §9.1: read `server/stack/*.py` routes and codes from the sibling `../../starfront` checkout and hold them to the hub profile; skipped with a note when the checkout is absent.
Verify: `node scripts/check-contract.mjs`

- [ ] T417 — Server, hub and pages on the home network
  Manual: `run_local.py --lan` and `collab-hub run` on the Mac; register the hub; `collab-uploader` to `http://<mac>:8800/stack/api`; the hub pulls, stacks and pushes; `http://<mac>:8800/stack/p/<slug>` renders in a browser on the Windows PC.

Phase 5 — the Starfront desktop (SF, §Phase 5)

- [ ] T501 — Settings sections `hub` and `volunteer`
  Files: SF `astrocontrol/config.py`
  Do: §5.1: both sections in `DEFAULTS` (unknown keys are dropped on save).
  Verify: `python3 tools/check_controls.py`

- [ ] T502 — `astrocontrol/hub.py`: supervising `collab-hub`
  Files: SF `astrocontrol/hub.py`, `astrocontrol/main.py` (CRLF), `tools/check_hub.py`
  Do: §5.2, §5.4: write the TOML, run `check`, start, tail the JSON log, poll status, restart with back-off, `caffeinate` on macOS, register with the server, mint enrol codes; the `/api/hub*` routes and the `/ws` summary.
  Verify: `python3 tools/check_hub.py`

- [ ] T503 — `astrocontrol/volunteer.py`: install a verified worker
  Files: SF `astrocontrol/volunteer.py`, `astrocontrol/main.py` (CRLF), `requirements.txt`, `packaging/starfront.spec`, `tools/check_volunteer.py`
  Do: §5.3, §5.4: manifest from the hub then the server; Ed25519 and SHA-256 checks; unpack; `enrol`; `service install`; status; update by copy-and-rename; the `/api/volunteer*` routes.
  Done when: a bad signature is refused and no unverified binary is ever run.
  Verify: `python3 tools/check_volunteer.py`

- [ ] T504 — The sequencer's busy file
  Files: SF `astrocontrol/sequencer.py`, `tools/check_night.py`
  Do: §5.3: touch `<data_root>/collab-worker/busy` at sequence start, remove at the end and on recovery exit.
  Verify: `python3 tools/check_night.py && python3 tools/check_recovery.py`

- [ ] T505 — The Collab tab: stacking fieldset, hub wizard, volunteer section
  Files: SF `astrocontrol/web/index.html` (CRLF), `astrocontrol/web/collab.js`, `astrocontrol/web/style.css`
  Do: §5.5: backend choice on the collaboration form; the hub section (folders, check, start/stop, register, enrol code, fleet, log); the volunteer section (hub, code, install/update/status, idle switches).
  Verify: `python3 tools/check_controls.py`

- [ ] T506 — README: "Live stacking at home"
  Files: SF `README.md` (CRLF)
  Do: §5.6: the design decisions (pull from the server, per-hub grants, the Windows logon task, the

user's own catalogue copy, what a volunteer's machine sees) and the Layout entries for the new modules.
Verify: `python3 -c "import sys; d=open('README.md','rb').read(); sys.exit(0 if b'Live stacking at home' in d and d.count(b'\n')==d.count(b'\r\n') else 1)"`

- [ ] T507 — A volunteer installs from the desktop
Manual: on the Windows PC, the Volunteer page installs from the Mac hub's manifest; the worker runs as a logon task and appears in the Mac's fleet table; it pauses while N.I.N.A. is open; `packaging/build.ps1` builds an installer that includes `cryptography` and the new pages.

Phase 6 — the N.I.N.A. plugin (SF, §Phase 6)

- [ ] T601 — Models, settings and link fields for a per-project backend
Files: SF `nina/src/Starfront.Collab/Server/Models.cs`, `Settings/StarfrontSettings.cs`, `Services/Stacking/StackingLinks.cs`
Do: §6.1: `StackingBinding` on the listing; `StackingHubTrustLan`, `StackingDefaultBackend`; link `Backend`, `ApiBase`, `LanApiBase`, `Grant`, `GrantExpiresUtc`, `HubId`.
Verify: `cd nina && dotnet build Starfront.sln -c Release`

- [ ] T602 — `StackingClients` cache, `GrantTokenStore` and `RoutingApiClient`
Files: SF `nina/src/Starfront.Collab/Services/Stacking/{StackingClients,GrantTokenStore,RoutingApiClient,StackingUploader}.cs`, `nina/tests/Starfront.Collab.Tests/`
Do: §6.2: a client per (base, token source); project-scoped calls routed by project, rig and library calls by the default backend (D001), device calls to the cloud; Core untouched.
Verify: `cd nina && dotnet build Starfront.sln -c Release && dotnet test tests/Starfront.Collab.Tests`

- [ ] T603 — The LAN short-cut
Files: SF `nina/src/Starfront.Collab/Services/Stacking/LanProbe.cs`, `StackingUploader.cs`, `nina/tests/Starfront.Collab.Tests/`
Do: §6.2: probe `{HubLanUrl}/api/hub/v1/health` with the hub id check; the effective base per link; a frame finishes on the base it started on.
Verify: `cd nina && dotnet test tests/Starfront.Collab.Tests`

- [ ] T604 — Joining hub projects without a Collab sign-in
Files: SF `nina/src/Starfront.Collab/Services/Stacking/StackingJoiner.cs`, `Ui/StackingOptionsVM.cs`, `Ui/OptionsTemplates.xaml`, `nina/tests/Starfront.Collab.Tests/`
Do: §6.3: the hub path of `ReconcileAsync`; sign-in required only for cloud links; the options text.
Verify: `cd nina && dotnet build Starfront.sln -c Release && dotnet test tests/Starfront.Collab.Tests`

- [ ] T605 — Checklist, SOURCE.md and change notes
Files: SF `nina/MANUAL_CHECKLIST.md`, `nina/src/RapidStackerCollab.Core/SOURCE.md`, `nina/README.md`, `nina/CHANGELOG.md`, `server/nina_plugin/CHANGELOG.md`
Do: §6.4, §6.5: the "Hub projects" checklist section; "Local differences: none" with the two upstream candidates; a draft changelog entry (the version number and release are the author's).
Verify: `cd nina && dotnet build Starfront.sln -c Release`

- [ ] T606 — The hub checklist in N.I.N.A.
  Manual: run `nina/MANUAL_CHECKLIST.md` "Hub projects" on the Windows PC against the home-network topology.

Phase 7 — local mode, docs, CI, the full night (§Phase 7)

- [ ] T701 — `tools/local_night.py`
  Files: SF `tools/local_night.py`
  Do: §7.2: start the local server, hub and a worker on the Mac; register the hub; bind a throw-away collaboration; print the URLs and an enrol code for the Windows PC; tail the logs; `--synthetic` feeds `collab-uploader` frames; `--exit-when-stacked` exits 0 once a live preview is published (the gate).
  Verify: `python3 tools/local_night.py --synthetic --exit-when-stacked`

- [ ] T702 — Documentation across both repositories
  Files: RSC `docs/runbooks/starfront-hub.md`, `CLAUDE.md`, `AGENTS.md`, `RAPID_STACKER_COLLAB_PLAN.md` (§2 and §7 pointers); SF `README.md` (CRLF), `server/README.md`
  Do: §7.2, §7.3: the final runbook and topology; coexistence with the AWS stack; the Starfront health route's `stackContract`. The loose `Astro/Running-the-servers-locally.md` gains a pointer to the runbook (it is unversioned).
  Verify: `node scripts/check-contract.mjs`

- [ ] T703 — Starfront CI that actually runs
  Files: SF `.github/workflows/` (moved from `nina/.github/workflows/`), `.github/workflows/python-checks.yml`
  Do: §7.4: move `ci.yml` and `release.yml` to the root with `working-directory: nina` and `paths: ['nina/**']`; run every `tools/check_*.py` on ubuntu and windows with Python 3.12. Can land at any time. (`CLAUDE.md` is git-ignored here; update the local copy's "Nothing runs CI" line.)
  Verify: `test -f .github/workflows/ci.yml && test -f .github/workflows/python-checks.yml && test ! -d nina/.github/workflows`

- [ ] T704 — Deleting a hub-backed collaboration
  Files: RSC `docs/runbooks/deleting-a-hub-project.md`, `worker/crates/hub/src/routes/admin.rs`; SF `server/app.py`, `server/hubs.py`, `tools/check_server.py`
  Do: §9.2 item 4, per D004: closing or deleting a collaboration files a delayed deletion on the hub and purges the relay copy; the runbook.
  Verify: `python3 tools/check_server.py`

- [ ] T705 — The full night
  Manual: `tools/local_night.py` on the Mac, then a real night from N.I.N.A. on the Windows PC: a light shows `accepted` in the dock and in the live preview within a minute, once over the LAN short-cut and once with the hub's LAN port blocked (server relay); gold publishes; the Discord webhook fires.